



UNIVERSITI MALAYSIA TERENGGANU
FACULTY OF COMPUTER SCIENCE AND MATHEMATICS

CSE3433

Software Architecture

Cloud-Native Learning Management System (LMS)
(Cloud-Native / Layered Architecture)

Prepared by:

Group No.5

No.	Name	Matric No.
1	NUR MAS QASRINA BINTI MAS JAYA	S74706
2	AIDA HUSNA BINTI MUHAMAD ZAIDI	S76103
3	NURSYAHEERA BINTI MOHD HISHAM	S76276

Prepared for:

DR MOHD NOR HASSAN

COMPUTER SCIENCE (SOFTWARE ENGINEERING) WITH HONOUR
SEMESTER II 2025/2026

Table of Contents

1. INTRODUCTION.....	3
2. PROBLEM DESCRIPTION.....	3
3. SYSTEM REQUIREMENTS.....	4
3.1 FUNCTIONAL REQUIREMENT.....	4
3.2 NON-FUNCTIONAL REQUIREMENT.....	5
4. ARCHITECTURE DESIGN.....	6
5. ARCHITECTURE DIAGRAMS.....	8
5.1 SYSTEM CONTEXT DIAGRAM.....	8
5.2 COMPONENT / SERVICES ARCHITECTURE DIAGRAM.....	9
5.3 INTERACTION DIAGRAM.....	11
5.4 DEPLOYMENT DIAGRAM.....	16
6. TECHNOLOGY STACK.....	17
7. PROTOTYPE IMPLEMENTATION.....	19
8. ARCHITECTURE EVALUATION.....	24
9. TEAM CONTRIBUTIONS.....	25
10. CONCLUSION AND REFLECTION.....	26

1. INTRODUCTION

This Cloud-Native Learning Management System (LMS) utilizing a Layered Architecture style is to address the critical infrastructure and performance challenges faced by modern educational platforms at Universiti Malaysia Terengganu (UMT). Traditional academic systems frequently suffer from severe latency, database lockups and complete runtime server failures during high-concurrency periods such as course registration weeks and synchronized examination windows.

To eliminate these infrastructural vulnerabilities, the proposed platform implements a Cloud-Native Managed Monolithic Model-View-Controller (MVC) model. It combines a structured Layered Architecture to isolate internal system concerns—specifically separating UI presentation (JSP), controller logic (Java Servlets) and data persistence (DAO Pattern) with Cloud-Native platform capabilities via Railway. By deploying this consolidated monolithic architecture onto a modern Platform-as-a-Service (PaaS) ecosystem, the system guarantees efficient traffic distribution, high availability, simplified dependency management and rapid horizontal scaling for all three primary user classes like Students, Lecturers and System Administrators.

2. PROBLEM DESCRIPTION

a) Scalability & Latency Bottlenecks

Conventionally hosted infrastructures often lack the capacity for automated resource allocation. In periods of intensive concurrent activity, such as final examination windows, the inability to scale horizontally leads to significant latency, unresponsive services, and overall performance decline.

b) Infrastructure Rigidity & Poor Layer Separation

Current learning platforms typically utilize legacy frameworks where essential functionalities, including user authentication, presentation layers, and database queries. The functionalities are tightly coupled within unoptimized codebases. Consequently, a single localized database

connection error or runtime exception within one module can compromise the entire application server runtime. This lack of strict logical layer separation complicates maintenance and increases the deployment risks associated with minor hotfixes.

c) Deployment and Accessibility Constraints

Traditional deployment workflows require tedious manual server configuration and downtime for every update. It is because the disruption to the academic calendar. Furthermore, many outdated learning systems fail to provide a modern, responsive, device-independent interface, which severely hinders the digital experience for students accessing learning resources via mobile hardware.

3. SYSTEM REQUIREMENTS

3.1 FUNCTIONAL REQUIREMENT

a. User Management

- FR - 1.1 : The system shall allow users to register an account by providing email, full name and password.
- FR - 1.2 : The system shall provide a secure login interface that authenticates users with encrypted credentials.
- FR - 1.3 : The system shall enforce Role Based Access Control, which ensures only authorized users can access specific pages such as the user list for admin.

b. Course Management

- FR - 2.1 : The system shall allow instructors to create new courses by entering a category, title and description.
- FR - 2.2 : The System shall support uploading course materials in multiple formats, including PDF documents and MP4 video files.
- FR - 2.3 : The system shall provide a search function that enables students to find courses using keywords or instructor names.

c. Assignment & Submission

- FR - 3.1 : The system shall allow instructors to set specific due dates and time limits for each assignment.
- FR - 3.2 : The system shall permit students to upload assignment files with a maximum size limit.
- FR - 3.3 : The system shall automatically block or reject a submission made after the deadline.

d. Notification

- FR - 4.1 : The system shall send automated notifications via email or dashboard when instructors post new course material.
- FR - 4.2 : The system shall send a push notification to student 24 hours before an assignment deadline.

e. Progress Tracking

- FR - 5.1 : The system shall update student a percentage complete each time they mark a lesson as finished.
- FR - 5.2 : The system shall display a visual progress bar on the student dashboard for every enrolled course.

3.2 NON-FUNCTIONAL REQUIREMENT

a. Availability

- NFR - 1.1 : The system shall maintain an uptime 99.9% ensuring student can reliably access course material and submit assignments at any time.
- NFR - 1.2 : The system shall implement an automated failover mechanism to guarantee service continuity in the event of a cloud server failure.

b. Scalability

- NFR - 2.1 : The system shall support horizontal scaling which enabling it to handle sudden increase in concurrent user without performance degradation.
- NFR - 2.2 : The database shall be capable of storing and retrieving data for up to 10,000 active users while maintaining acceptable latency levels.

c. Performance

- NFR - 3.1 : The system shall deliver a response time of less than 2 seconds for standard page loads under normal traffic conditions.
- NFR - 3.2 : File upload and download processes shall be optimized like a 20MB assignment file is completed within 10 seconds.

d. Security

- NFR - 4.1 : all sensitive data, include user password and personal information shall be encrypted or equivalent industry standard before storage in the database.
- NFR - 4.2 : The system shall enforce HTTP for all data transmission to safeguard against unauthorized interception of student and instructor information.

e. Maintainability & usability

- NFR - 5.1 : The system architecture shall be modular which allows for seamless updates or the addition of new features without disrupting existing services.
- NFR - 5.2 : The user interface shall be fully responsive by ensuring consistent functionality across smartphone, desktops and tablet,

4. ARCHITECTURE DESIGN

Table 4 : The Services and Description for Cloud-Native Layered Monolithic Architecture

Table 4 show the system implements a Cloud-Native Layered Monolithic Architecture structured around the classic Model-View-Controller (MVC) architectural pattern.

No.	Services	Description
1	Presentation Layer	A device-agnostic, responsive client-side interface built using JavaServer Pages (JSP), HTML5 and Tailwind CSS. It handles

		user-centric interactions and renders data dynamically on the server side based on active session states without requiring heavy client-side frameworks.
2	Controller & Business Logic Layer (Controller)	Functions as the dynamic routing core managed by a series of stateless Java Servlets (LoginServlet, CreateCourseServlet, etc.) running inside a cloud-hosted Tomcat servlet container. This layer orchestrates core business logic, intercepts incoming HTTP POST/GET requests, enforces session-based Role-Based Access Control (RBAC), and manages execution workflows.
3	Data Access Layer (Model / DAO)	Employs the Data Access Object (DAO) design pattern powered by robust JDBC (Java Database Connectivity). This layer decouples the upper application logic from the raw database queries. It utilizes secure SQL PreparedStatement to handle transactional operations while completely preventing SQL injection vulnerabilities.
4	Data Layer (Persistence)	Utilizes a Centralized Relational Database Management System (MySQL) hosted as an independent, fully-managed managed cloud instance on Railway. This tier enforces strict data normalization, referential integrity (foreign key constraints), and optimized indexing across user credentials, course catalogs, and progress metrics.

5. ARCHITECTURE DIAGRAMS

5.1 SYSTEM CONTEXT DIAGRAM

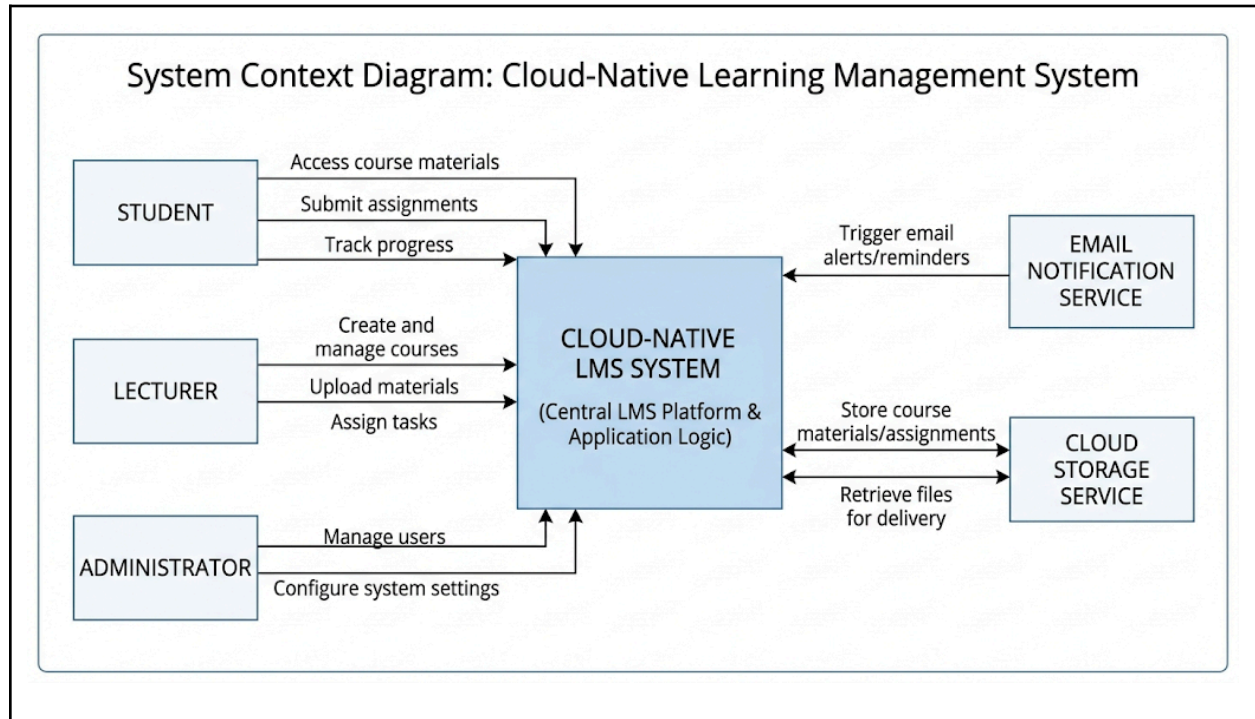


Figure 3 : System Context Diagram for Cloud-Native Learning Management System.

The System Context Diagram provides a high-level architectural abstraction of the external interface boundaries between the Cloud-Native LMS and its primary actor classes, specifically Students, Lecturers and System Administrators. This visualization highlights the system boundaries where user interactions are consolidated. In a modern cloud-native deployment ecosystem such as Railway. The diagram outlines how user traffic is captured and how internal subsystems delegate secondary operational tasks such as localized file management and automated alert triggers without breaking the application's unified execution context. The diagram illustrates the orchestration of user-centric flows essential to the platform's daily academic operations by encapsulating internal subsystem complexity.

5.2 COMPONENT / SERVICES ARCHITECTURE DIAGRAM

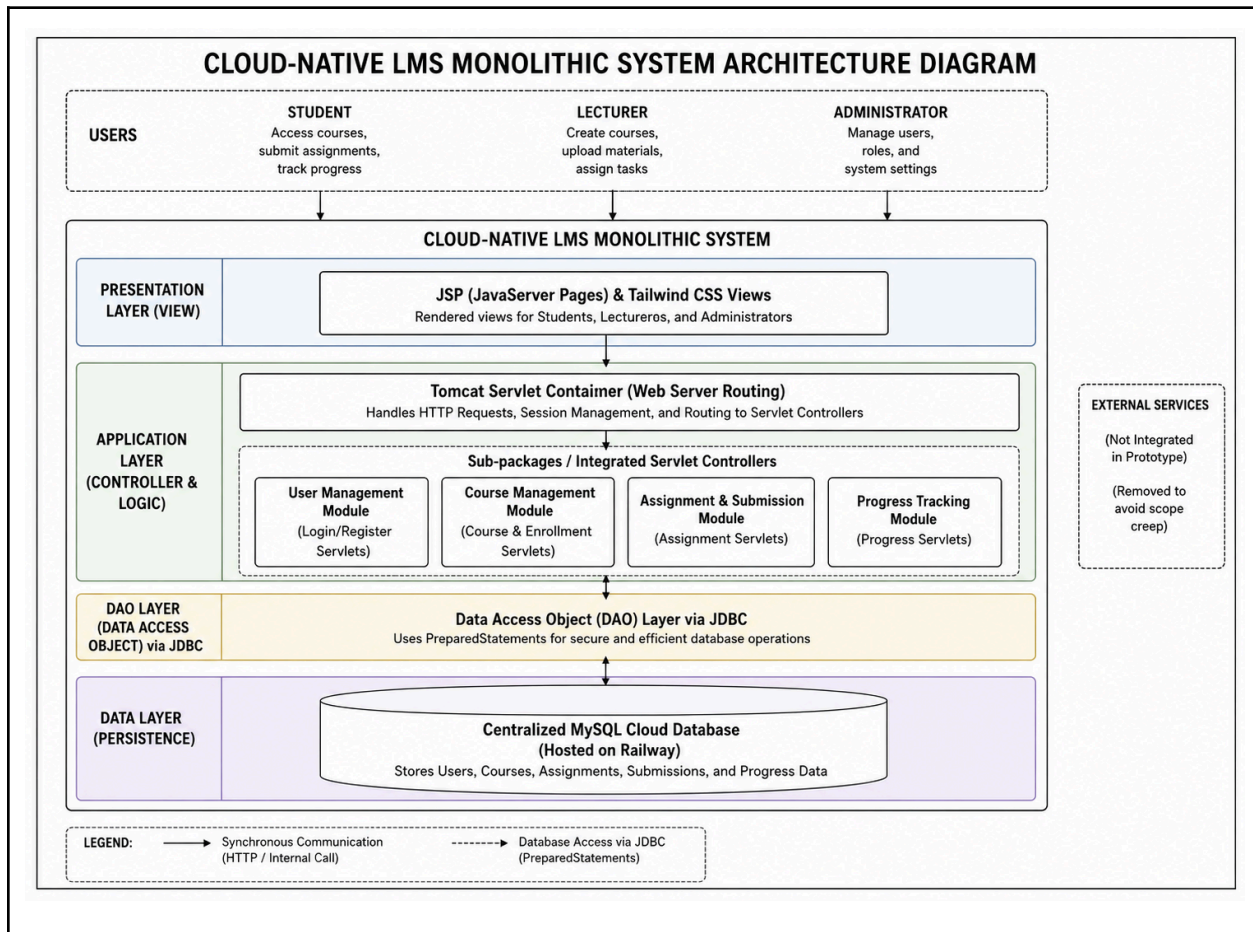


Figure 4 : Component Diagram for Cloud-Native LMS Microservices Architecture Diagram

The internal architectural configuration of the Cloud-Native LMS, predicated on a structured layered monolithic design is visually represented within the Component Architecture Diagram in Figure 4. This cohesive framework is strategically partitioned into Presentation, Application, Data Access and Data tiers where each tier is assigned specific isolated operational mandates to ensure strict logical separation of concerns under the Model-View-Controller (MVC) paradigm.

Presentation Layer (View) managed via a device-agnostic web frontend built with JavaServer Pages (JSP) and Tailwind CSS. This layer orchestrates all user-centric graphical interactions ranging from course selection to assignment uploads. It acts as a strict thin-client layer, delegating all state mutations and transaction requests down to the application tier via server-side form actions and standard HTTP methods.

Application Layer (Controller & Business Logic) operates inside a cloud-hosted Tomcat servlet container. Unlike decoupled microservices that introduce distributed networking overhead, this architecture consolidates domain capabilities into specialized, integrated Java Servlets (LoginServlet, RegisterServlet, CreateCourseServlet). These controllers handle endpoint routing, extract request parameters, enforce session-based authorization filters, and manage the application's procedural business rules.

Data Access Layer (Model) implements the Data Access Object (DAO) pattern using native JDBC (Java Database Connectivity). This explicit layer bridges the gap between raw application logic and the persistence layer. It ensures secure transactional handling, query optimization and compile-time validation while mitigating security risks such as SQL injection by using SQL PreparedStatements.

Data Layer (Persistence) utilizes a Centralized Relational Database Management System (MySQL) fully hosted as an independent instance on the Railway cloud platform. Rather than splitting data across distributed schemas, this centralized persistence engine enforces strict referential integrity, absolute data consistency, and transactional safety (ACID compliance) across all tables, including user records, course metadata, and completion scores.

In summary, this layered monolithic approach optimizes engineering overhead, simplifies deployment pipelines, and guarantees tight cohesion between components. When combined with the automated scaling, memory allocation, and git-integrated deployment capabilities of the Railway Platform-as-a-Service (PaaS) infrastructure, the platform is highly optimized to handle contemporary academic concurrency demands at UMT.

5.3 INTERACTION DIAGRAM

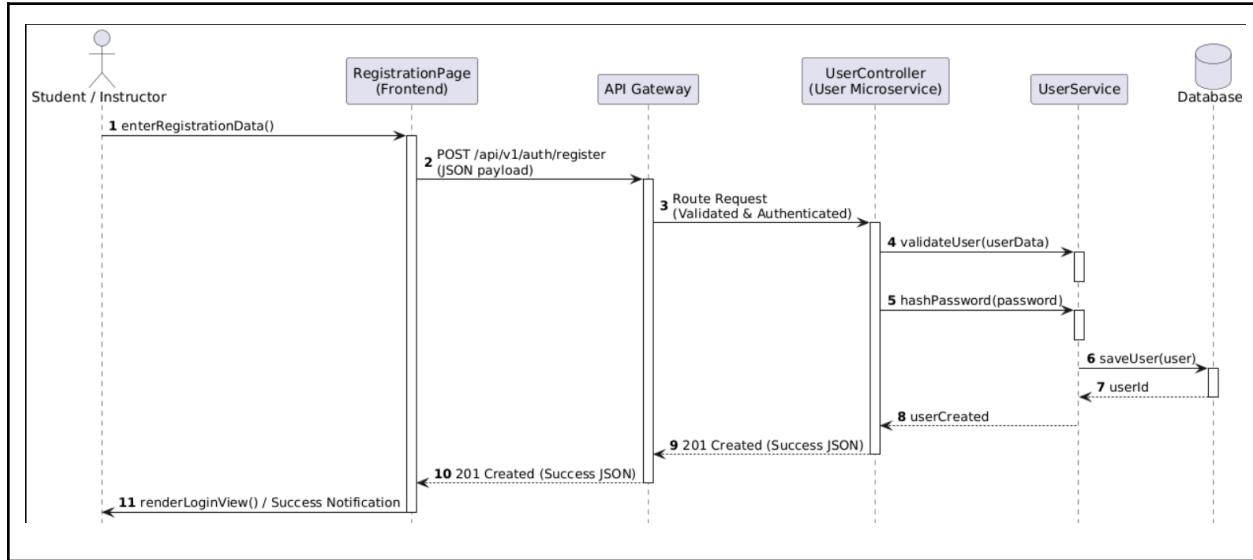


Figure 5 : User Management (User Registration) Sequence Flow

The cloud-native sequence diagram for the User Registration module illustrates a fully decoupled, API-first communication lifecycle. The process initiates when a Student or Instructor submits credential inputs through the responsive RegistrationPage presentation interface. This client payload is dispatched as a synchronous HTTPS POST network request to the API Gateway. The gateway executes edge validation and policies before securely proxy-routing the JSON payload to the UserController inside the independent User Management Microservice. The internal UserService validates the entity structure , cryptographically hashes the raw text password strings , and commits the user state object to the isolated Database domain. Upon generating a success response code (201 Created) , the confirmation cascades back through the gateway layer to the client-side UI, triggering a state update to prompt the user to log in safely.

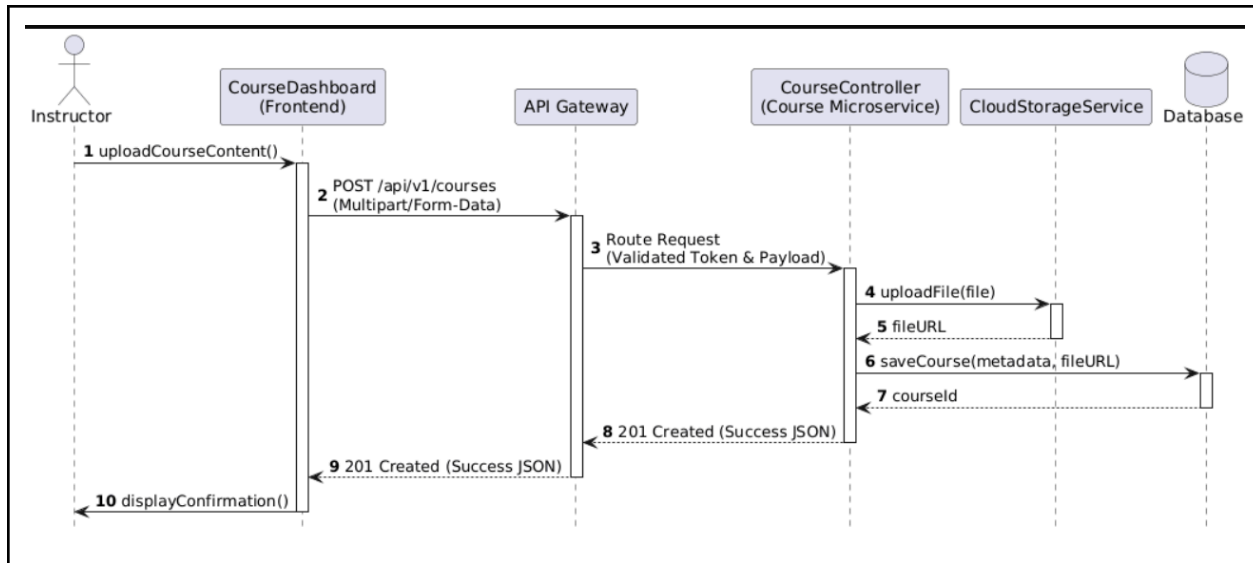


Figure 6: Course Management (Create New Course) Sequence Flow

The architectural sequence for the Course Management module demonstrates a decoupled, event-driven lifecycle for the ingestion of academic assets. This process commences when an *Instructor* submits media files and curricular metadata through the responsive *CourseDashboard* presentation layer. This client-side interaction is dispatched as a secure HTTPS multipart request to the centralized API Gateway, which performs edge orchestration including request throttling and identity verification. The gateway then proxy-routes the network payload to the *CourseController* situated within the isolated Course Management Microservice. This service component offloads binary content to distributed Cloud Storage for optimized persistence while retaining a unique resource locator string. This metadata is subsequently committed to the domain-specific Database to ensure state consistency. Upon receipt of a transactional success code, the confirmation signal cascades back to the frontend interface, triggering a visual status update to notify the user of a successful upload.

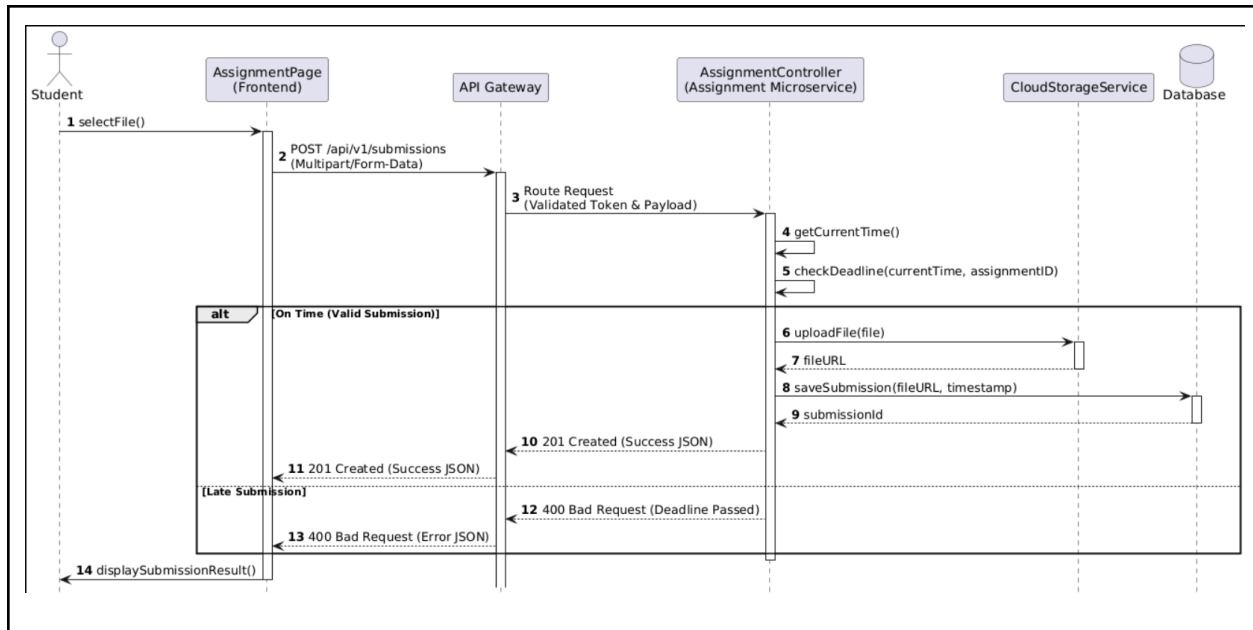


Figure 7: Assignment & Submission (Student Upload) Sequence Flow

The architectural sequence for the Assignment & Submission module demonstrates a decoupled, asynchronous lifecycle for the ingestion and verification of student artifacts. This process commences when a *Student* submits digital files through the responsive *AssignmentPage* presentation layer. This client-side interaction is dispatched as a secure HTTPS multipart request to the centralized API Gateway, which performs edge orchestration including traffic policy enforcement. The gateway then proxy-routes the network payload to the *AssignmentController* situated within the isolated Assignment Management Microservice. This service component executes an immediate temporal validation, comparing the current system timestamp against the curricular deadline metadata cached in memory. Upon validation, the controller offloads binary content to distributed Cloud Storage for optimized persistence while retrieving a unique resource locator string. This metadata, alongside the submission record, is subsequently committed to the domain-specific Database to ensure state consistency. Upon receipt of a transactional success code, the confirmation signal cascades back through the gateway to the frontend interface, triggering a visual status update to notify the user of a successful submission.

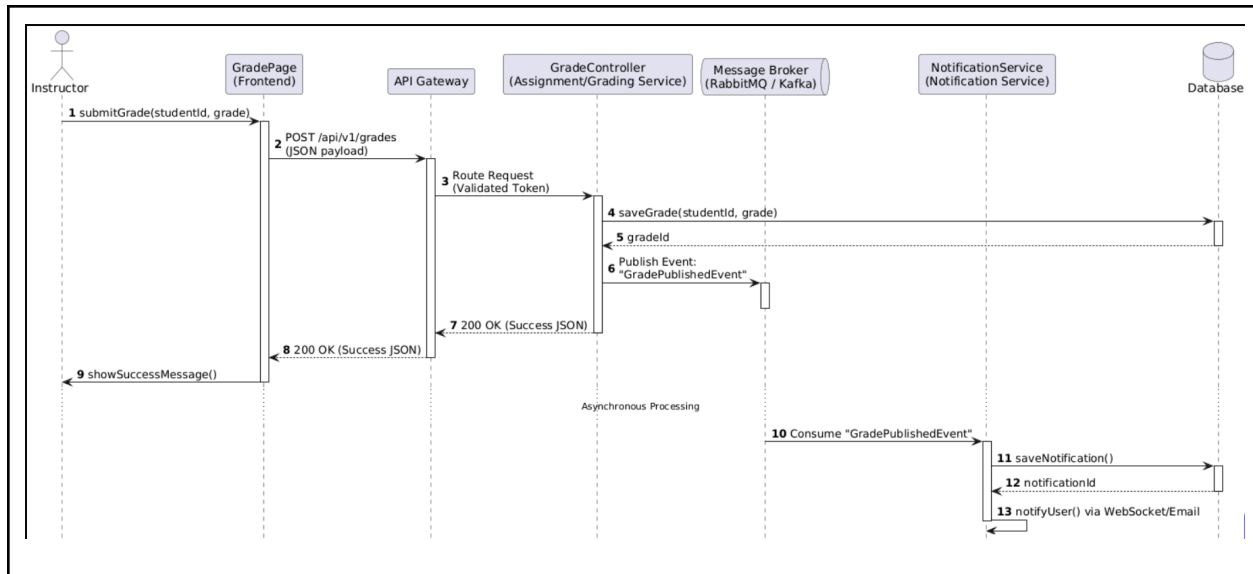


Figure 8: Notification Module (Trigger Grade Alert) Sequence Flow

The cloud-native sequence diagram for the Notification Module illustrates an asynchronous, event-driven orchestration pattern engineered for fault isolation. The process initiates when an Instructor dispatches a student evaluation evaluation score via the GradePage presentation layer. The interface submits an HTTP POST payload through the API Gateway, which validates and routes the traffic to the GradeController inside the independent Assignment/Grading Microservice. Once the record is successfully committed to the database , the grading service publishes a stateless GradePublishedEvent message packet to the distributed Message Broker (RabbitMQ/Kafka) and immediately delivers a successful execution response back to the client interface. This non-blocking pattern decouples the services completely; the NotificationService independently consumes the event payload from the message queue asynchronously, persists the alert configuration to the data layer, and handles real-time delivery channels (such as WebSockets or email push mechanisms) to the end user without impacting the core processing metrics of the grading application layer.

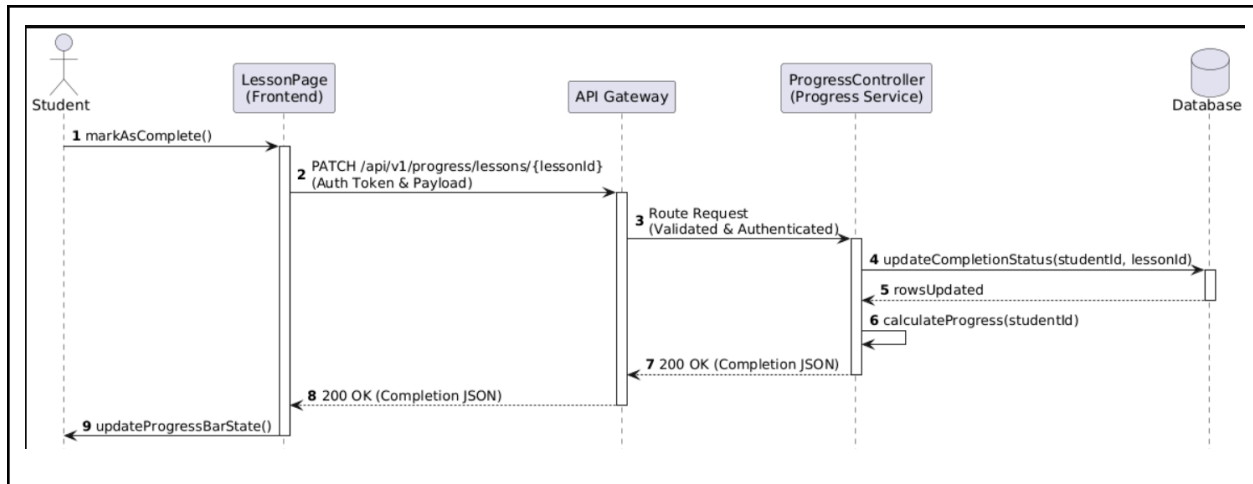


Figure 9: Process Tracking (Update Completion) Sequence Flow

The cloud-native sequence diagram for the Progress Tracking module outlines the runtime lifecycle of a user state completion transaction. The process initiates when a Student interacts with the responsive LessonPage client layer to trigger a completion state change. This client event is transformed into an asynchronous HTTP PATCH request and dispatched directly to the API Gateway. The gateway enforces authorization validation filters before routing the request packet to the ProgressController inside the decoupled Progress Tracking Microservice. The service triggers an isolated transaction against the domain Database to record the lesson completion status. Upon processing the update, the microservice internal logic handles the aggregation metrics to compute the student's total updated course progress metrics. This calculated state is returned as a structured JSON object inside a standard HTTP 200 OK network response, cascading back to the presentation layer where the frontend state manager dynamically repaints the visual progress dashboard components without requiring a full page refresh.

5.4 DEPLOYMENT DIAGRAM

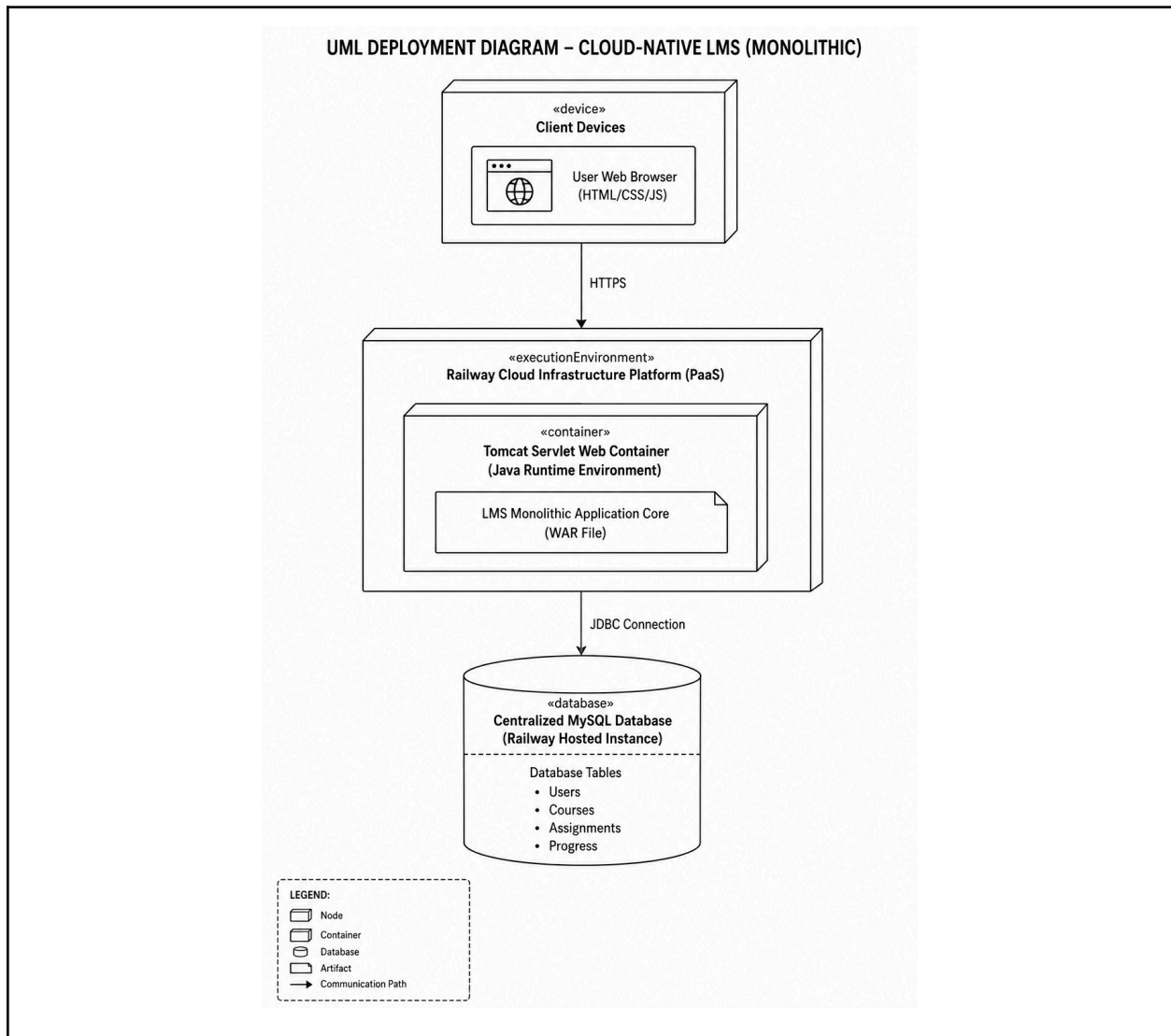


Figure 10: Deployment Diagram of Cloud-Native Learning Management System (LMS)

The deployment diagram in Figure 10 maps out the physical distribution and runtime environment of the LMS components across an elastic that fully-managed cloud infrastructure ecosystem. Unlike distributed microservices clusters that demand high network configuration and orchestration overhead, this deployment model emphasizes resource efficiency and rapid delivery by leveraging a consolidated Platform-as-a-Service (PaaS) topology via Railway.

Client Boundary is a End-users such as Students, Instructors, and Administrators interface with the system using standard modern web browsers on cross-platform consumer hardware devices.

Inbound public application traffic is strictly secured over HTTPS, routing requests directly to the application's cloud-native ingress interface.

Application Compute Tier (Railway Environment) which is the entire application logic is encapsulated into a single unified deployment unit operating within a managed Tomcat Web Server or Servlet Container. Railway automatically provisions the underlying Java Runtime Environment (JRE) that offering features such as automated Git-integrated CI/CD pipelines, container monitoring, and vertical resource scaling. This node processes incoming requests, manages state-driven Java Servlets, and dynamically compiles JSP presentation views.

Database & Persistence Tier to prevent architectural anomalies and data synchronization bottlenecks, the data layer strictly enforces a centralized relational structure. It allocates an independent, fully-managed MySQL Cloud Database Instance running natively on Railway's dedicated database infrastructure. The application container establishes a resilient network bridge to this instance via secure JDBC (Java Database Connectivity) on Port 3306. This design allows the core system to execute lightweight transaction queries rapidly while remaining robust, highly available, and entirely fault-tolerant against hardware server failovers.

6. TECHNOLOGY STACK

Table 6 : The Technology Selected for each Component and Engineering Justification also The Role in The System.

The Table 6 show the implementation of the Cloud-Native Learning Management System (LMS) leverages a stable, enterprise-grade open-source technology stack.

Component / Tier	Technology Selected	Engineering Justification & Role in System
Presentation Layer (View)	JavaServer Pages (JSP), HTML5, & Tailwind CSS	Used to build a fully responsive, device-agnostic user interface. JSP enables server-side rendering (SSR), allowing dynamic academic data (student

		course lists, assignment status) to be injected directly into the HTML document before delivery to the client browser that optimizing page-load speeds.
Controller / Routing Tier	Java Servlets (Jakarta EE / Java EE Core)	Serves as the centralized control mechanism inside the Tomcat runtime environment. Web requests are dynamically intercepted and mapped (@WebServlet annotations) to individual servlet classes such as LoginServlet and CreateCourseServlet to execute specific backend business logic and session validations.
Data Access Layer (Model)	JDBC (Java Database Connectivity) via DAO Pattern	Acts as the explicit data access bridge. It decouples the core application logic from raw SQL execution. The use of native JDBC PreparedStatement ensures all database operations are pre-compiled and heavily sanitized, preventing malicious SQL injection attacks.
Database Management (Data Layer)	Railway Platform-as-a-Service (PaaS)	Serves as the modern cloud execution environment. It acts as the Cloud-Native engine by hosting both the monolithic Tomcat web app container and the MySQL database instance. Railway automates Git-integrated CI/CD pipelines, handles runtime log aggregation, manages environment secrets and provides horizontal resource scaling.

7. PROTOTYPE IMPLEMENTATION

This section presents the visual representation of the developed system, demonstrating the functionality and user interface design of the platform.

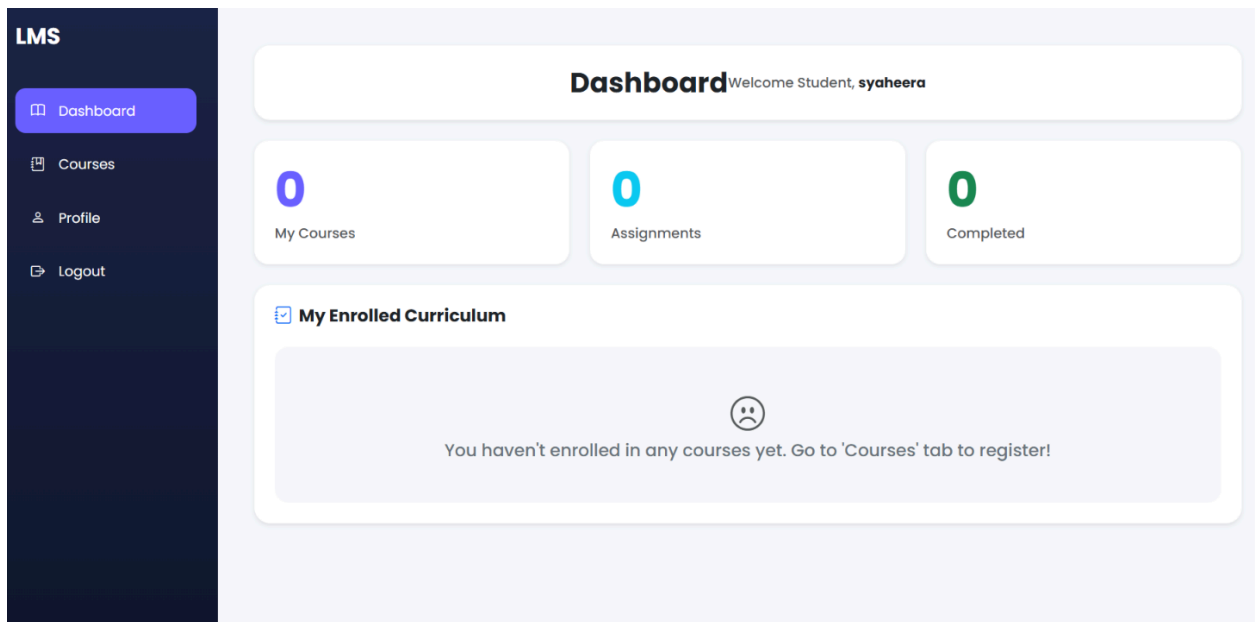


Figure 7.1: Student Dashboard Interface

The Student Dashboard acts as the primary user interface (UI) designed for personalized learning management. It integrates real-time data from the enrollments, assignments, and submissions tables in the `lms_db` database. The visual hierarchy is structured to prioritize user productivity, where the 'My Courses' count provides an immediate snapshot of the student's current workload. This section utilizes server-side rendering (JSP) to fetch data dynamically based on the student's session ID, ensuring that the information displayed is always specific and relevant to the logged-in user.

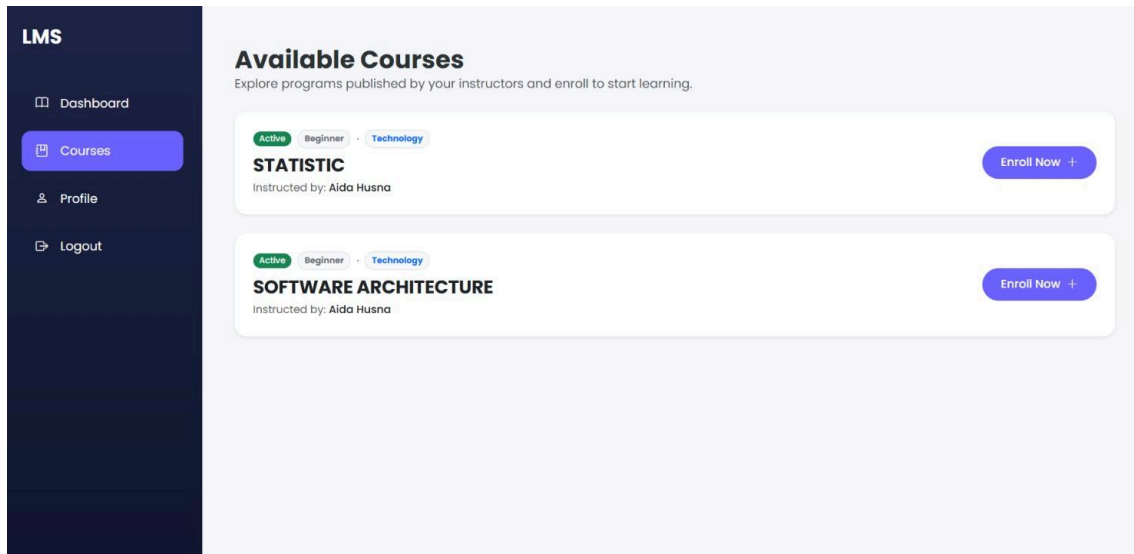


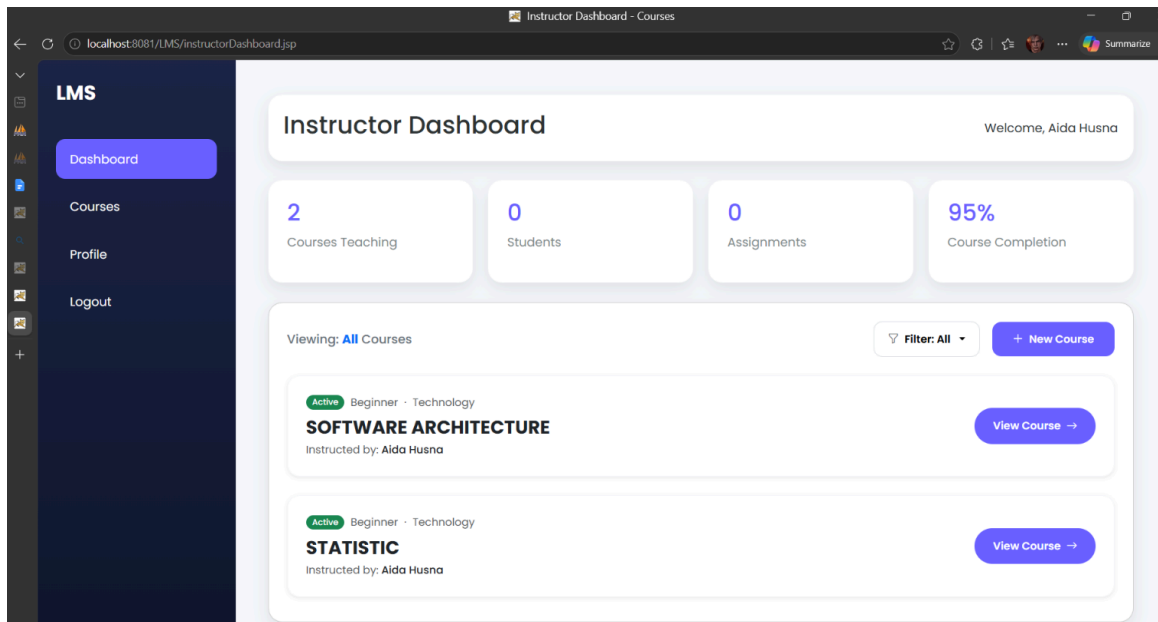
Figure 7.2: Course Selection and Enrollment

The Course Selection page implements a dynamic filtering mechanism that allows students to browse through the curriculum. Behind the scenes, the enrollment process utilizes a doGet servlet handler that processes the courseId and studentId parameters. This figure highlights the user's journey from course discovery to registration. The implementation includes server-side validation to prevent duplicate enrollments, ensuring database integrity and providing immediate visual feedback to the student via URL parameters upon a successful transaction

The figure displays two screenshots of the LMS user authentication interface. The top screenshot shows the 'Register' form, which includes fields for 'No. Matrik' (with a placeholder 'Contoh: S76276'), 'Full Name', 'Username' (with a placeholder 'aide@gmail.com'), 'Email', 'No. Telefon', and 'Password'. The bottom screenshot shows the 'Welcome Back!' login form, which includes fields for 'No. Matrik' (with a placeholder 'TEST999'), 'Password', and a 'Log in As:' dropdown menu set to 'Student'. Both forms have a 'Login' button and a link to 'Register here'.

Figure 7.3: User Authentication (Login/Register)

The authentication module is the gateway to the system, ensuring that access is restricted to authorized personnel only. This figure displays the front-end forms for both registration and login, which are connected to the RegisterServlet and LoginServlet respectively. The logic flow utilizes a UserDao class to handle secure database interactions. By separating the roles into 'Student', 'Instructor', and 'Admin', the system enforces role-based access control (RBAC) at the servlet level, verifying user credentials against the users table before granting access to the respective dashboards.



← Back to Dashboard

Create New Course

Course Title *

e.g. Intro to Web Development

Description

Instructor Name

Aida Husna

Category

Technology

Level

Beginner

Description

Figure 7.4: Instructor Dashboard & Course Management

This figure illustrates the Instructor Dashboard, a centralized control panel for managing academic content. The instructor is provided with the capability to create, update, and delete courses. When an instructor clicks 'Add Course', the data is processed by CreateCourseServlet and stored in the courses table within the lms_db database. This module demonstrates the system's dynamic content management capability, allowing instructors to maintain up-to-date course materials without requiring manual database intervention.

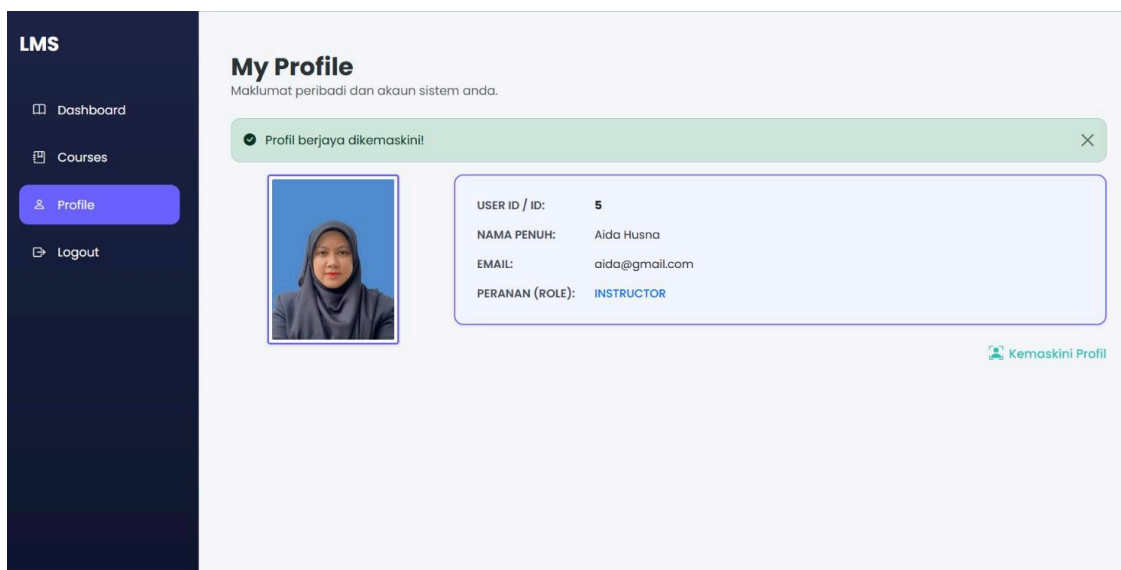


Figure 7.5: User Profile View

The User Profile view provides a personalized summary of the student or instructor's details. The data displayed is retrieved dynamically from the users table based on the primary key, id (Matric Number). This figure demonstrates the system's capability for data persistence, ensuring that user-specific information remains accurate throughout the session.

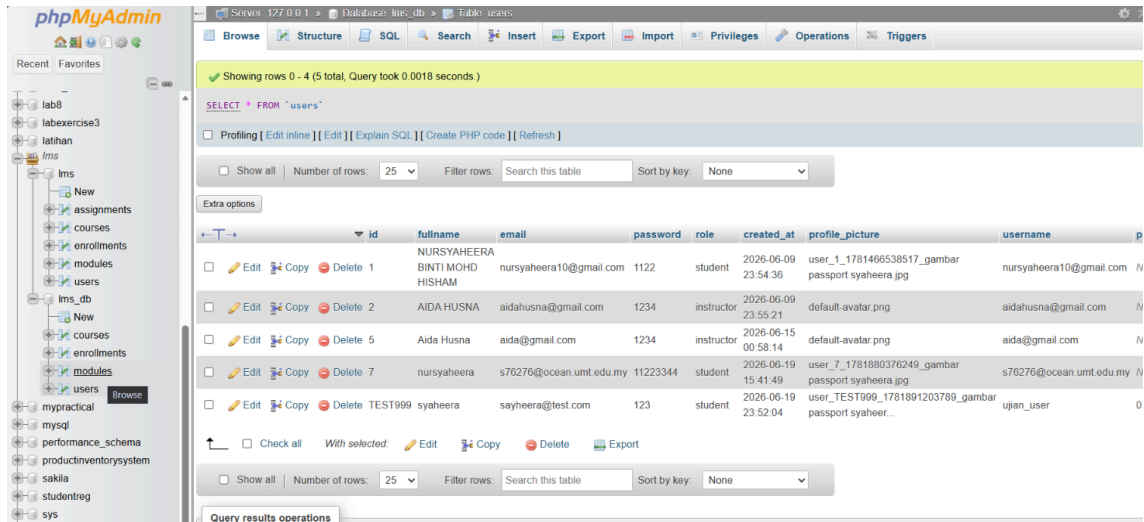


Figure 7.6: Backend Logic & Database Integration

This figure captures the core logic layer of the application, specifically the DAO (Data Access Object) classes. By employing the Model-View-Controller (MVC) pattern, we isolate the database connection logic from the presentation layer. The snippet highlights the use of PreparedStatement to execute INSERT and SELECT queries, which acts as a secure 'bridge' between the Java Servlet controllers and the MySQL database. This architecture ensures that the application is modular, secure, and easier to debug

8. ARCHITECTURE EVALUATION

Table 8: The Architecture Criteria and Thier Evaluation

The system architecture was evaluated based on its performance, reliability, and ability to handle the specific requirements of an academic learning management environment.

Criteria	Evaluation
Scalability	The use of MySQL as a relational database allows for stable data growth. As student records increase, the normalized structure ensures consistent performance.
Reliability	The use of Railway (PaaS) for the application layer enables automated deployments and self-healing properties, ensuring the LMS remains operational with minimal downtime.
Integrity	By employing SQL PreparedStatements, the architecture effectively prevents SQL injection attacks, ensuring that

	user data, grades, and permissions remain secure.
Cloud-Native Storage	Integration with AWS services provides enterprise-grade durability, automated backups, and geo-redundancy, safeguarding data against hardware failures.
Infrastructure	The combination of Railway for rapid application delivery and AWS for robust database management optimizes the CI/CD pipeline, reducing manual configuration overhead.

The system's cloud-native architecture leverages Railway to decouple application logic from infrastructure, enabling independent scaling and rapid deployment cycles while maintaining high stability. Complementing this, AWS provides a robust storage layer that offloads critical maintenance such as security patching and disaster recovery, ensuring consistent performance even during traffic spikes. By utilizing an MVC pattern for modularity and maintainability, the architecture successfully delivers a stable, transactional-based system that is both secure and highly extensible for future integration with NoSQL databases.

9. TEAM CONTRIBUTIONS

Table 9: Team Members Roles & Responsibilities

Table 9 presents the roles assigned to each team member along with their respective responsibilities throughout the project development.

No.	Name	Role	Responsibilities
1.	Nur Mas Qasrina Binti Mas Jaya	Project Manager	Directs the comprehensive project framework, architecting the cloud-native infrastructure mapping and Model-View-Controller (MVC) layered system configurations. This role involves modeling complex component relationships, logical layer boundaries, and managing the unified

			integration of all architectural tiers.
2.	Aida Husna Binti Muhamad Zaidi	Backend & Database Engineer	Designs the centralized relational database schemas at the Data Layer implements core business logic workflows using Java Servlets at the Application or Controller Layer, constructs the Data Access Object (DAO) pattern using secure JDBC and configures cloud-native environment synchronization for Railway deployment. .
3.	Nursyaheera Binti Mohd Hisham	Frontend & UI/UX Engineer	Develops the device-agnostic, responsive client-side interface using JavaServer Pages (JSP) and Tailwind CSS at the Presentation Layer, handles server-side dynamic data rendering injection, configures form-action submission routings to the respective Java Servlets and optimizes cross-browser interface consistency.

10. CONCLUSION AND REFLECTION

Transitioning from tightly-coupled legacy systems to a well-structured, Cloud-Native Layered Monolithic Architecture under the Model-View-Controller (MVC) paradigm is the optimal engineering strategy to permanently mitigate the platform instability typically found in university-wide LMS platforms. This project provides foundational software engineering insights into structural layer separation, secure data-access encapsulation via the DAO pattern and automated deployment pipelines within a Platform-as-a-Service (PaaS) ecosystem. By consolidating the application core into a single and highly cohesive

unit while offloading runtime resource scaling to Railway's cloud infrastructure. The finalized architecture successfully ensures that UMT students and faculty members can depend on a highly available, robust, secure and future-ready digital learning space.